Solution: Problem Challenge 3: Employee Free Time

Problem Statement

For 'K' employees, we are given a list of intervals representing each employee's working hours. Our goal is to determine if there is a **free interval which is common to all employees**. You can assume that each list of employee working hours is sorted on the start time.

Example 1:

```
Input: Employee Working Hours=[[[1,3], [5,6]], [[2,3], [6,8]]]
Output: [3,5]
Explanation: All the employees are free between [3,5].
```

Example 2:

```
Input: Employee Working Hours=[[[1,3], [9,12]], [[2,4]], [[6,8]]]
Output: [4,6], [8,9]
Explanation: All employees are free between [4,6] and [8,9].
```

Example 3:

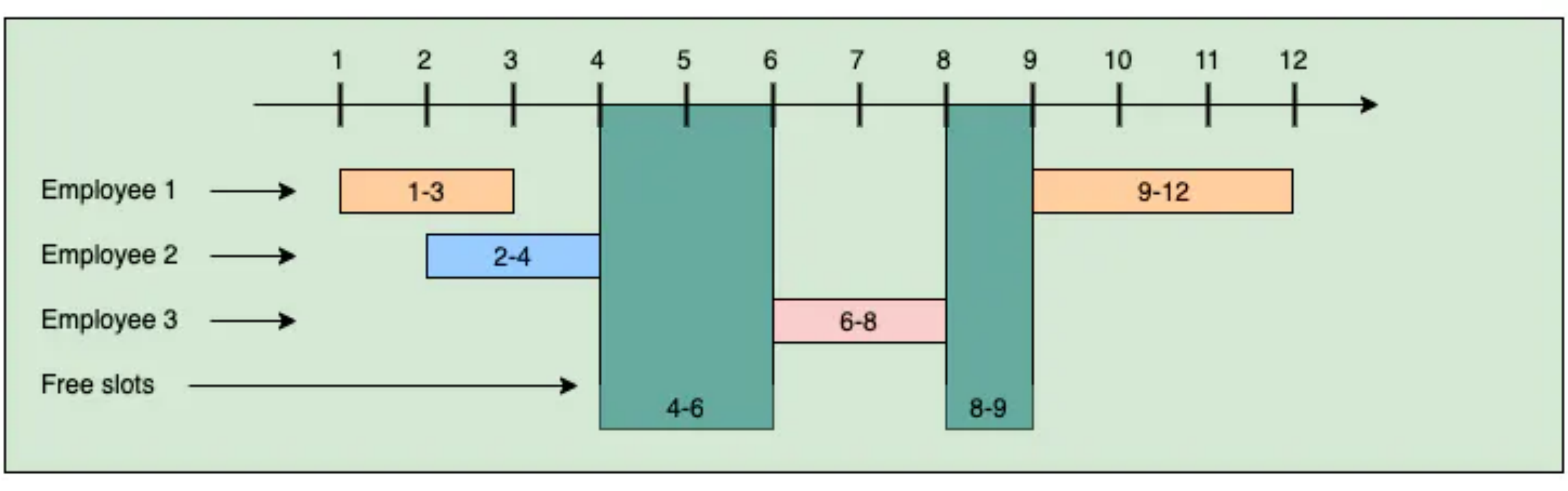
```
Input: Employee Working Hours=[[[1,3]], [[2,4]], [[3,5], [7,9]]]
Output: [5,7]
Explanation: All employees are free between [5,7].
```

Solution

This problem follows the **Merge Intervals pattern**.

Let's take the above-mentioned example (2) and visually draw it:

```
Input: Employee Working Hours=[[[1,3], [9,12]], [[2,4]], [[6,8]]]
Output: [4,6], [8,9]
```



One simple solution can be to put all employees' working hours in a list and sort them on the start time. Then we can iterate through the list to find the gaps.

Let's dig deeper.

Sorting the intervals of the above example will give us:

```
[1,3], [2,4], [6,8], [9,12]
```

We can now iterate through these intervals, and whenever we find non-overlapping intervals (e.g., [2,4] and [6,8]), we can calculate a free interval (e.g., [4,6]). This algorithm will take $O(N * logN)$ time, where 'N' is the total number of intervals. This time is needed because we need to sort all the intervals. The space complexity will be O(N), which is needed for sorting.

Can we find a better solution?

Using a Heap to Sort the Intervals

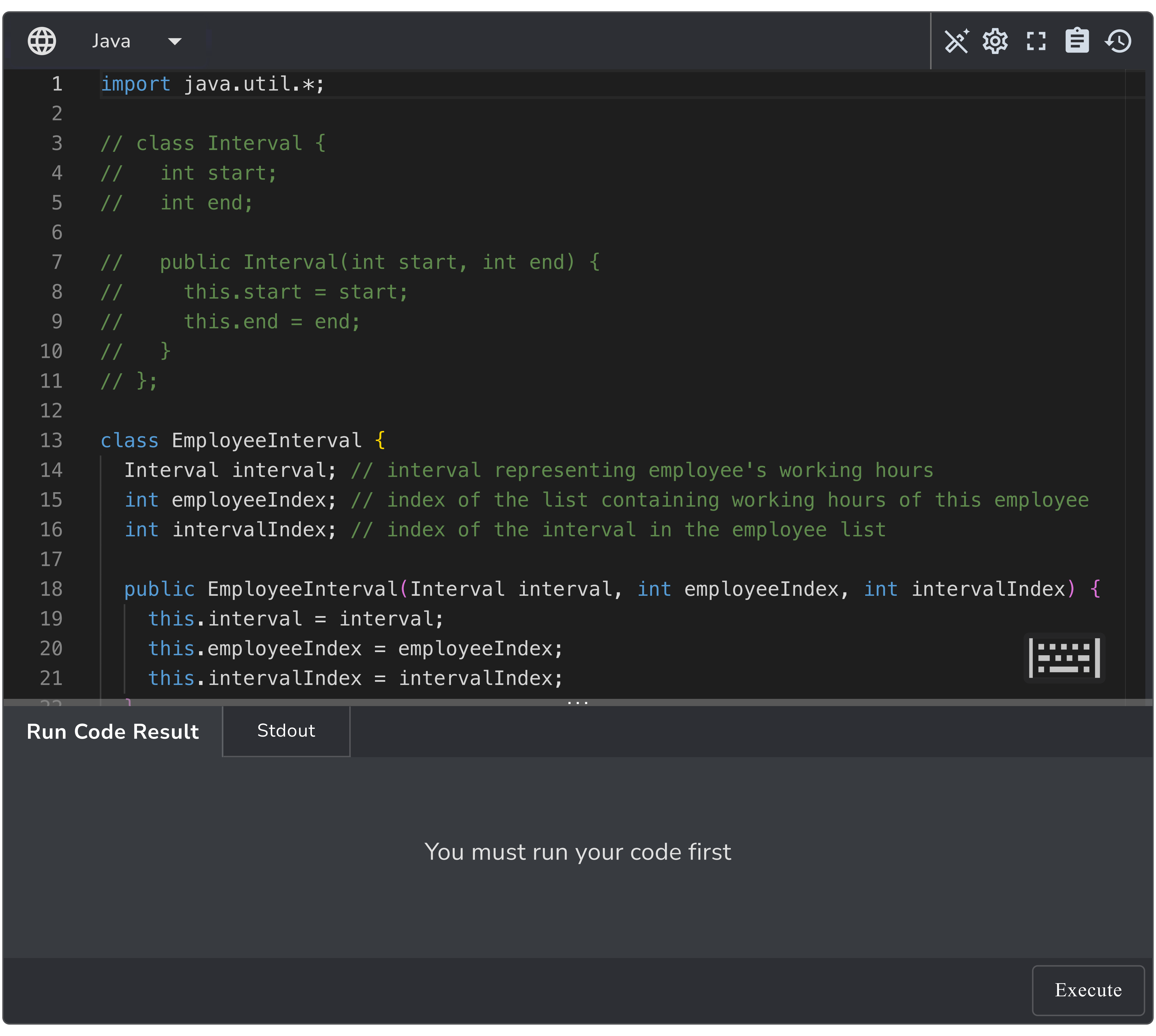
One fact that we are not utilizing is that each employee list is individually sorted!

How about we take the first interval of each employee and insert it in a **Min Heap**. This **Min Heap** can always give us the interval with the smallest start time. Once we have the smallest start-time interval, we can then compare it with the next smallest start-time interval (again from the **Heap**) to find the gap. This interval comparison is similar to what we suggested in the previous approach.

Whenever we take an interval out of the **Min Heap**, we can insert the same employee's next interval in the heap. This also means that we need to know which interval belongs to which employee.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

// class Interval {
//   int start;
//   int end;

//   public Interval(int start, int end) {
//     this.start = start;
//     this.end = end;
//   }
// };

class EmployeeInterval {
  Interval interval; // interval representing employee's working hours
  int employeeIndex; // index of the list containing working hours of this employee
  int intervalIndex; // index of the interval in the employee list

  public EmployeeInterval(Interval interval, int employeeIndex, int intervalIndex) {
    this.interval = interval;
    this.employeeIndex = employeeIndex;
    this.intervalIndex = intervalIndex;
```

You must run your code first

Time Complexity

The above algorithm's time complexity is $O(N * logK)$, where 'N' is the total number of intervals, and 'K' is the total number of employees. This is because we are iterating through the intervals only once (which will take O(N)), and every time we process an interval, we remove (and can insert) one interval in the **Min Heap**, (which will take $O(logK)$). At any time, the heap will not have more than 'K' elements.

Space Complexity

The space complexity of the above algorithm will be $O(K)$ as at any time, the heap will not have more than K elements.

← Previous
Problem Challenge 3: Employee Free Time (Hard)

Next →
Introduction To Cyclic Sort Pattern

☐ Mark as Completed